

# Übungsblatt 1b

---

## Web Engineering Grundlagen

5430UE Web and Data Engineering

31. März 2026

Prof. Dr. Michael Granitzer

# Lernziele

- **Grundprinzipien** und **Kategorien** des Web Engineering kennen und abgrenzen
- Die **Evolution des Webs** (1.0, 2.0, 3.0) verstehen
- **CSS-Selektoren** sicher für das Frontend-Styling einsetzen können
- **Media Queries** für responsive Layouts anwenden

# Modul A — Web Engineering Grundlagen

---

# Übung 1b.A.1: Kategorien von Web-Anwendungen

Ordnen Sie folgende Systeme einer Web-Anwendungskategorie zu (statisch, dynamisch server-seitig, Single-Page Application, Web-Service/API):

| System                                                             | Kategorie |
|--------------------------------------------------------------------|-----------|
| Universitäts-Stundenplan als HTML-Seite, wöchentlich neu generiert | ?         |
| Online-Banking-Portal mit Echtzeit-Kontoübersicht                  | ?         |
| Wetter-REST-API, die JSON zurückgibt                               | ?         |
| Produktkatalog mit Suchfilter ohne Seitenneuladen                  | ?         |

Begründen Sie je eine Zuordnung kurz.

# Übung 1b.A.1: Kategorien — Lösung

| System                    | Kategorie                                                                                  |
|---------------------------|--------------------------------------------------------------------------------------------|
| Stundenplan HTML          | <b>Statisch</b> (vorgerendert, kein serverseitiger Code pro Request)                       |
| Online-Banking            | <b>Dynamisch server-seitig</b> (Session, Authentifizierung, Datenbankabfragen per Request) |
| Wetter-REST-API           | <b>Web-Service/API</b> (kein HTML, maschinenlesbare Daten)                                 |
| Produktkatalog mit Filter | <b>Single-Page Application</b> (JS übernimmt Routing und Datenabfragen im Browser)         |

**Begründung Wetter-API:** Eine API liefert strukturierte Daten (JSON/XML) für Maschinen, nicht für Menschen. Die Darstellung obliegt dem konsumierenden Client.

# Übung 1b.A.2: Grundprinzipien des Web Engineering

Web Engineering ist die Anwendung systematischer und quantifizierbarer Ansätze (Konzepte, Methoden, Techniken, Werkzeuge).

1. Erläutern Sie die **Grundprinzipien** in eigenen Worten.
2. Welche **Ziele** verfolgt Web Engineering (Ziele und Qualitätsmerkmale)?

# Übung 1b.A.2: Grundprinzipien — Lösung

**Definition:** Web Engineering ist die systematische Disziplin zur Entwicklung von Web-Anwendungen.

## Grundprinzipien:

- Klar definierte **Ziele und Anforderungen**
- Systematische Entwicklung in **Phasen**
- Kontinuierliche **Überwachung** des gesamten Entwicklungsprozesses
- Planung der **Wartung** (Post-Launch-Plan)

**Ziele:** Qualität, Wartbarkeit, Skalierbarkeit, Sicherheit, Usability, Wiederverwendbarkeit — bei vertretbaren Kosten.

# Übung 1b.A.3: Kategorien nach Kappel

Erläutern Sie kurz die Hauptunterschiede und geben Sie zu jeder Kategorie ein konkretes Beispiel:

| Kategorie           | Charakteristik | Beispiel |
|---------------------|----------------|----------|
| Dokumentenzentriert | ?              | ?        |
| Interaktiv          | ?              | ?        |
| Transaktional       | ?              | ?        |
| Workflow-basiert    | ?              | ?        |
| Kollaborativ        | ?              | ?        |
| Portalorientiert    | ?              | ?        |
| Ubiquitär           | ?              | ?        |
| Social Web          | ?              | ?        |
| Semantisches Web    | ?              | ?        |



# Übung 1b.A.3: Kategorien — Lösung

| Kategorie           | Charakteristik                               | Beispiel                         |
|---------------------|----------------------------------------------|----------------------------------|
| Dokumentenzentriert | Statische Inhalte, kaum Interaktion          | Firmen-Homepage                  |
| Interaktiv          | Auswahl, Navigation, dynamische Antworten    | News-Site, Fahrplanauskunft      |
| Transaktional       | Zustandsverändernde Operationen (Konsistenz) | Onlineshop, Online-Banking       |
| Workflow-basiert    | Mehrstufige Prozesse, Akteure/Rollen         | E-Government, Patienten-Workflow |
| Kollaborativ        | Mehrere Nutzer arbeiten gleichzeitig         | Wiki, gemeinsame Dokumente       |
| Portalorientiert    | Aggregation heterogener Dienste              | Unternehmensportal               |
| Ubiquitär           | Personalisiert, kontextabhängig              | Wetter-App                       |
| Social Web          | Nutzer-erzeugte Inhalte, Vernetzung          | Blogs, Social-Media              |
| Semantisches Web    | Maschinenverarbeitbare Daten                 | Recommender-System               |

**Hinweis:** Reale Anwendungen sind meist Mischformen (z.B. Amazon).

# Übung 1b.A.4: Web Engineering — Ziele und Prinzipien

Betrachten Sie die systematische Disziplin des Web Engineering.

1. Was unterscheidet **Web Engineering** von klassischer Software-Entwicklung?
2. Nennen Sie drei typische Qualitätsziele, die für Web-Anwendungen besonders kritisch sind.
3. Welche Probleme entstehen langfristig ohne Versionskontrolle und ohne Tests?

# Übung 1b.A.4: Ziele und Prinzipien — Lösung

1. **Web Engineering** behandelt die Besonderheiten: heterogene Clients, zustandsloses HTTP, öffentliche Erreichbarkeit, schnelle Releasezyklen, hohe Nutzerzahlen. Klassische SE optimiert oft für kontrollierte Deployment-Umfelder.
2. Typische Web-Qualitätsziele:
  - **Skalierbarkeit:** Millionen gleichzeitiger Nutzer
  - **Accessibility/Usability:** Barrierefreiheit auf allen Geräten
  - **Performance/Ladezeit:** Kritisch für die Nutzerbindung
3. **Langfristig:** Technische Schulden akkumulieren, Regression bei Änderungen, keine Basis für Compliance, schwieriges Onboarding neuer Entwickler.

# Übung 1b.A.5: Evolution des Web

Das Web hat sich in Generationen entwickelt:

| Merkmal               | Web 1.0 | Web 2.0 | Web 3.0 |
|-----------------------|---------|---------|---------|
| Inhaltsproduktion     | ?       | ?       | ?       |
| Nutzerinteraktion     | ?       | ?       | ?       |
| Typische Technologien | ?       | ?       | ?       |
| Beispiel-Anwendung    | ?       | ?       | ?       |

Füllen Sie die Tabelle aus. Welches Web-Paradigma dominiert heute?

# Übung 1b.A.5: Evolution des Web — Lösung

| Merkmal               | Web 1.0            | Web 2.0                         | Web 3.0                          |
|-----------------------|--------------------|---------------------------------|----------------------------------|
| Inhaltsproduktion     | Nur Betreiber      | Nutzer generieren Inhalte (UGC) | Dezentralisiert, maschinenlesbar |
| Nutzerinteraktion     | Passiv (Lesen)     | Aktiv (Kommentieren, Teilen)    | Semantisch, personalisiert       |
| Typische Technologien | Statisches HTML    | AJAX, Web-APIs, Blogs           | Linked Data, RDF, KI-Integration |
| Beispiel-Anwendung    | Broschüren-Website | Facebook, YouTube               | KI-Assistenten                   |

**Heute dominiert** Web 2.0 (interaktive SPAs, REST-APIs). Web-3.0-Elemente (semantische Metadaten) werden zunehmend integriert.

# Modul B — CSS & Frontend Basics

---

# Übung 1b.B.1: CSS Dinner — Selektoren

Bearbeiten Sie die **32 Level** aus dem CSS Dinner unter [flukeout.github.io](https://flukeout.github.io).

Diese Übung schärft die Grundlagen für CSS-Selektoren, die Sie im Frontend-Modul intensiv brauchen werden.

**Reflexion:** Welche Selektor-Typen haben Sie bisher noch nie verwendet? Welche fanden Sie am schwersten?

# Übung 1b.B.1: CSS Dinner — Hinweise

Wichtige Selektor-Typen:

- **Typ:** `plate`, `bento`
- **Klasse:** `.small` | **ID:** `#fancy`
- **Kombinatoren:** `A B` (Nachfahre), `A > B` (Kind), `A + B` (Nachbar), `A ~ B` (Geschwister)
- **Pseudo-Klassen:** `:first-child`, `:last-child`, `:nth-child(n)`, `:not()`, `:empty`
- **Attribut-Selektoren:** `[for]`, `[for^="val"]`

Erfahrungsgemäß sind `:nth-child` und die Geschwister-Kombinatoren am schwierigsten.



# Modul C — CSS Code lesen

---

# Übung 1b.C.1: CSS-Selektoren — TODO-Liste

Betrachten Sie einen HTML-Ausschnitt einer TODO-Liste. Ziel ist es, per CSS:

1. Schriftgröße aller `li` ohne Klasse `styles` auf 18px zu setzen.
2. Das zweite Element jeder Liste auszublenden.
3. *Unbedingt* (strong) rot zu färben.
4. Den Header (h1) zu transformieren.
5. Selektoren mit `for^="basic"` hervorzuheben.

# Übung 1b.C.1: CSS-Selektoren — Lösung

```
li:not(.styles)      { font-size: 18px; }  
li:nth-child(2)      { display: none; }  
strong               { color: red; }  
h1                   { transform: rotateZ(10deg) translate(20px, 40px); }  
li[for^="basic"]     { text-decoration: line-through; }
```

## Erklärung:

- `:not(.styles)` schließt Listeneinträge mit Klasse `styles` aus.
- `:nth-child(2)` greift jeweils das zweite Geschwisterkind.
- `[for^="basic"]` matcht alle Attribute, die mit `basic` **beginnen**.

# Übung 1b.C.2: Media Query

Erstellen Sie eine Prüfung auf Bildschirmgröße ( $< 600$  Pixel), um die Schriftgröße allgemein auf 10px zu verkleinern.

# Übung 1b.C.2: Media Query — Lösung

```
@media screen and (max-width: 599px) {  
  body { font-size: 10px; }  
}
```

## Erklärung:

- @media screen schränkt die Regel auf Bildschirme ein.
- (max-width: 599px) greift, wenn das Viewport schmaler als 600 Pixel ist.
- Innerhalb des Blocks gelten die Regeln nur unter dieser Bedingung.

# Materialien zum Übungsblatt

Zum Üben steht eine **leere HTML-Demo-Datei** bereit:

- [demo-empty.html](#) — leeres HTML-Gerüst zum Experimentieren

*Alle Materialien finden Sie auch zentral auf der **Kursseite** unter Materials → Download.*